# 학습 노트 (5). 에뮬레이션 킷 실습과 다섯 주의 정리

오픈소스 반도체 설계 및 제작 종합설계 (2026 Fall)

공과대학 기계공학부 안준영　학번 2024-11078　2026 년 10 월 23 일

## 들어가며

지난주에 받은 FPGA 검증 킷으로 Q.13 의 에뮬레이션과 프로토타이핑을 실습했다. 그동안 배운 내용을 짧게 정리하고, 보드를 돌리면서 겪은 일과 앞으로 내 칩으로 만들어 보고 싶은 것을 적었다.

## 1. 다섯 주 동안 배운 것

주차별로 한 일과 기억에 남은 것을 TABLE 1 에 정리했다.

TABLE 1. 주차별 실습과 남은 생각

| 주차 | 실습 | 남은 생각 |
|---|---|---|
| 1 | 남의 칩(ATmega328P)으로 탁구 게임기 실행 | 소프트웨어는 한 줄씩 실행되고 회로는 동시에 움직인다. 공 크기를 빼먹은 충돌 판정 버그를 찾았다. |
| 2 | 바이브 코딩 우주 침략자, SystemC 테스트벤치 이식, Makefile 수정 | 인공지능이 짠 코드도 특정 도구 버전과 환경에서만 돌아간다. |
| 3 | Q.1 과 Q.4 의 그래픽 처리, 스프라이트, 키보드 입력 | 사람이 키를 누르는 시간, 화면 한 장의 시간, 클럭 한 번의 시간을 맞춰야 한다. |
| 4 | Q.14 Talks 의 면적 견적, RTL 다섯 버전 비교 | 인공지능이 낸 견적과 수정 코드를 사람이 다시 확인해야 했다. |
| 5 | FPGA 에뮬레이션과 프로토타이핑 | 같은 RTL 을 시뮬레이터, FPGA, 실제 클럭 위에서 차례로 돌려 보았다. |

돌아보면 매주 같은 순서로 공부했다. C++나 아두이노로 먼저 돌려 보고, 베릴로그로 옮긴 다음, 같은 테스트벤치로 확인했다. 이번 주에 처음으로 그 베릴로그가 FPGA 위에서 실제 회로로 돌았다.

## 2. 보드를 돌리기까지

### 2.1 실습 환경

킷은 FPGA(Cyclone IV E, EP4CE6E22C8)가 달린 PSCE-TRANS 보드, Raspberry Pi Pico 와 OLED 가 달린 PSCE-MI 보드, FPGA 에 파일을 올리는 JTAG 케이블로 되어 있다(Fig. 1). FPGA 모듈의 USB-C 는 전원만 넣는 용도다.

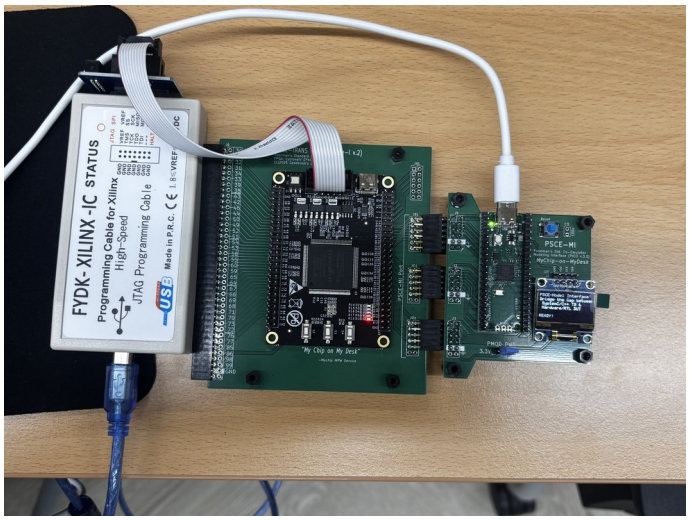



Fig. 1. 에뮬레이션 검증 킷. 왼쪽부터 JTAG 케이블, PSCE-TRANS(FPGA), PSCE-MI(Pico 와 OLED).

제일 먼저 막힌 건 Quartus 였다. Quartus 는 x86 컴퓨터에서만 돌아가서 그동안 쓰던 맥(ARM)의 우분투 가상 머신에는 설치되지 않았다. 그래서 x86 리눅스에 Quartus Prime Lite 25.1 과 Cyclone IV 장치 파일을 설치하고, Q.13 의 최종 RTL(4_Ok)로 FPGA 파일을 만들었다. Pico 펌웨어는 arduino-cli 에 RP2040 코어와 U8g2 라이브러리를 깔아서 빌드했다. 맥은 USB 포트가 모자라서 보드 연결과 실행은 윈도우 PC 의 WSL(Ubuntu 24.04)에서 했다. usbipd 로 Pico 와 JTAG 케이블을 WSL 로 넘기니 lsusb 에 둘 다 잡혔다(Fig. 2).

```
mecameca@DESKTOP-OL4SI7G:~$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 0403:6014 Future Technology Devices International, Ltd FT232H Single HS USB-UART/FIFO IC
Bus 001 Device 003: ID 2e8a:000a Raspberry Pi Pico
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
mecameca@DESKTOP-OL4SI7G:~$ ls /dev/ttyACM*
/dev/ttyACM0
mecameca@DESKTOP-OL4SI7G:~$
```

Fig. 2. WSL 에서 확인한 USB 장치. JTAG 케이블(FT232H, 0403:6014)과 Pico(2e8a:000a), Pico 의 직렬 포트 /dev/ttyACM0.

빌드 결과는 TABLE 2 에 정리했다. 에뮬레이션용은 게임 RTL 을 래퍼로 감싼 회로이고, 프로토타이핑용은 래퍼 없이 게임기 핀을 그대로 밖으로 뺀 회로다. 둘 다 FPGA 의 14 퍼센트 정도만 썼다.

TABLE 2. Quartus 빌드 결과 (EP4CE6E22C8, RTL 4_Ok)

| 항목 | 에뮬레이션용 | 프로토타이핑용 |
|---|---|---|
| 논리 소자 | 903 / 6,272 | 870 / 6,272 |
| 레지스터 | 180 | 150 |
| 핀 | 24 / 92 | 21 / 92 |
| 최대 동작 주파수 | 70.45MHz | 75.39MHz |
| Pico 펌웨어 크기 | 72,692 바이트 | 72,140 바이트 |

## 2.2 FPGA 를 알아보지 못한 openFPGALoader

FPGA 에 파일을 올리려고 openFPGALoader 를 실행했더니 Unknown device with IDCODE 0x020f10dd 라는 에러가 났다. 처음에는 케이블 문제인 줄 알았는데, 다시 보니 케이블은 FPGA 의 ID 까지 읽어 오고 있었다. JTAG 는

칩마다 정해진 IDCODE 로 어떤 칩인지 알아보는데, 우분투 저장소에 있던 옛날 버전은 이 칩의 ID 를 몰랐다. 최신 소스(v1.2.0)를 받아 다시 빌드하니 EP4CE6 으로 인식하고 파일이 올라갔다.

## 3. 에뮬레이션

Pico 에 모델링 인터페이스 펌웨어를 넣고 FPGA 에 에뮬레이션용 파일을 올린 다음, 시뮬레이션 때 쓰던 SystemC 테스트벤치를 그대로 실행했다. Connection established 가 뜨고 게임 화면이 그려지기 시작했다(Fig. 3). 보드의 OLED 에도 같은 화면이 나왔다(Fig. 4).

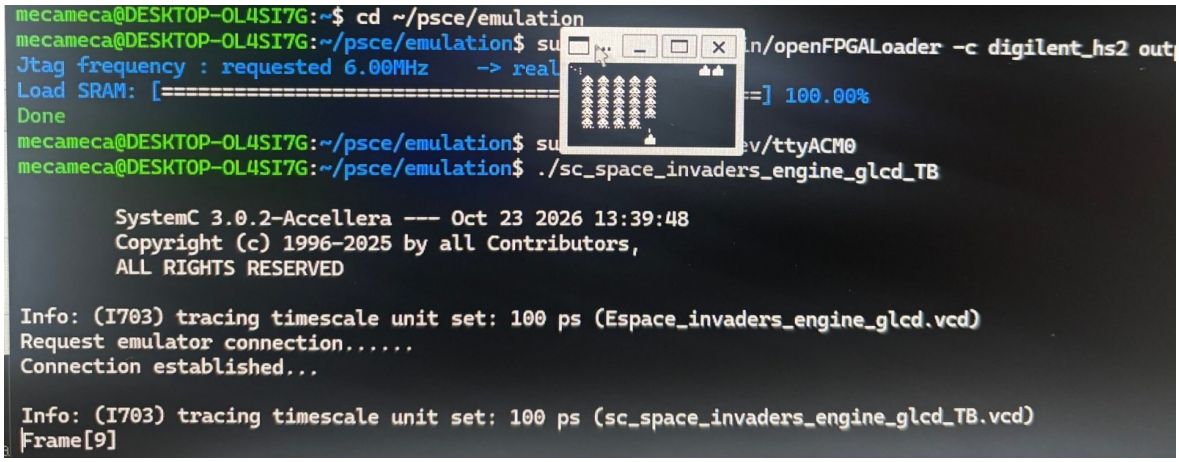



Fig. 3. 에뮬레이션 실행. FPGA 에 파일을 올리고(Load SRAM 100%) 테스트벤치가 Pico 와 연결되어 게임 화면을 그린다.

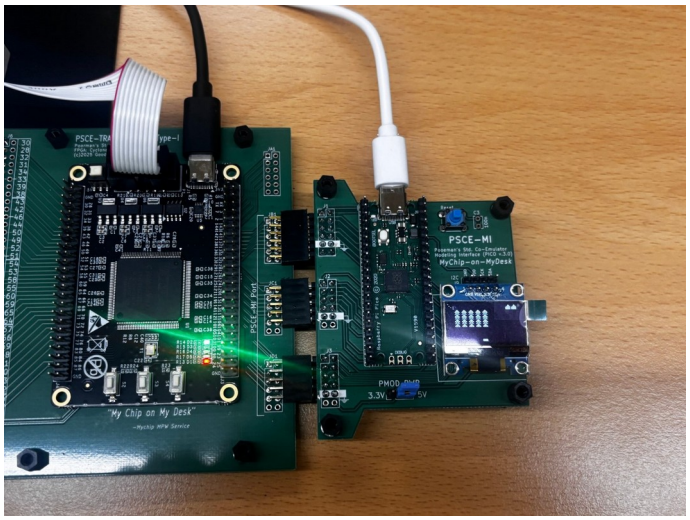



Fig. 4. 에뮬레이션 중인 킷. PSCE-MI 의 OLED 에 외계인 대형이 보인다.

그런데 화면이 아주 느리게 바뀌었다. 몇 분을 돌렸는데 프레임은 80 장까지만 넘어갔다. 코드를 보니 이유가 있었다. 테스트벤치는 DUT 의 클럭 한 번마다 USB 로 입력 1 바이트를 보내고 출력 2 바이트를 받는다. 화면 한 장이 128 x 64 = 8,192 클럭이니 한 장을 그리려면 USB 를 8,192 번 오가야 한다. USB 는 장치를 1 초에 수천 번 정도만 확인하기 때문에 한 장에 몇 초씩 걸린다. 에뮬레이션은 시뮬레이션보다 빠르다고 배웠는데, PC 와 FPGA 를 잇는 통로가 느리면 오히려 더 느렸다.

## 4. 프로토타이핑

이번에는 Pico 에 프로토타이핑 펌웨어를, FPGA 에 프로토타이핑용 파일을 올리고 키보드 입력 프로그램을 실행했다(Fig. 5). 이 프로그램은 화면을 그리지 않고 방향키와 스페이스바 상태만 Pico 로 보낸다. 게임은 보드의 OLED 에서 돌아갔고, 키를 누르면 포대가 움직였다.

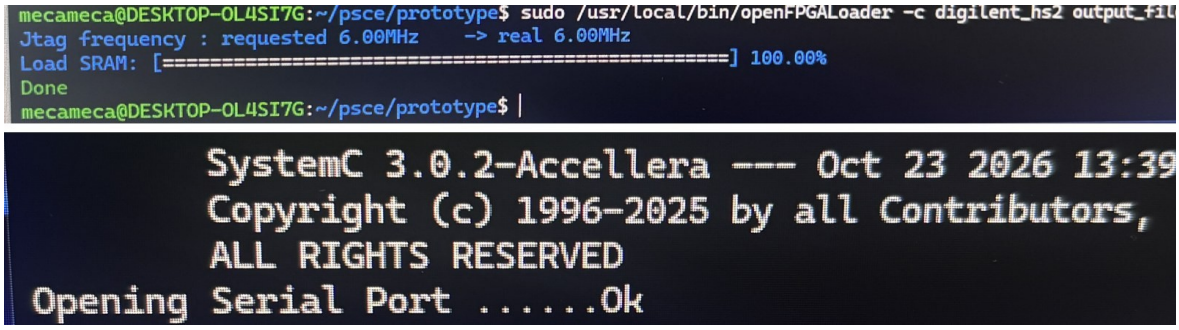



Fig. 5. 프로토타이핑 실행. 위는 프로토타이핑용 파일을 FPGA 에 올린 화면, 아래는 키보드 입력 프로그램이 직렬 포트에 연결된 화면.

에뮬레이션과 다른 점은 클럭을 Pico 가 직접 만든다는 것이다. 펌웨어를 보면 RP2040 의 PWM 으로 100kHz 클럭을 만들어 GPIO28 로 FPGA 에 넣는다. 이 속도면 화면 한 장(8,192 클럭)에 약 0.08 초, 1 초에 12 장 정도다. 외계인을 움직이는 game_tick 은 16,000 클럭마다 한 번이라 외계인은 0.16 초에 한 칸씩 간다.

4 주차 노트에서 틀린 게 하나 있었다. 4 번 RTL 의 주석에 16,000 이 1MHz 를 60Hz 로 나눈 값이라고 적혀 있어서 검증 보드가 1MHz 로 돈다고 썼는데, 실제 펌웨어의 클럭은 100kHz 였다. 주석에 적힌 것보다 열 배 느리게 돌고 있었던 것이다. 주석만 보고 쓸 게 아니라 클럭을 만드는 펌웨어까지 봤어야 했다.

## 5. 시뮬레이션, 에뮬레이션, 프로토타이핑

세 가지를 직접 해 본 결과를 TABLE 3 에 비교했다.

TABLE 3. 세 가지 검증 방식의 비교

| 항목 | 시뮬레이션 | 에뮬레이션(CA) | 프로토타이핑 |
|---|---|---|---|
| DUT | Verilator 가 만든 C++ 모델 | FPGA 에 합성된 회로 | FPGA 에 합성된 회로 |
| 클럭 | SystemC 커널이 생성 | 테스트벤치가 매 클럭 USB 로 진행 | Pico PWM 100kHz |
| 테스트벤치 | SystemC | 시뮬레이션과 같은 SystemC | 키보드 입력 프로그램 |
| 화면 | PC 의 SDL 창 | PC 의 SDL 창과 OLED | OLED |
| 속도 | PC 성능에 따름 | 가장 느림 (한 장에 수 초) | 실시간에 가까움 |
| 내부 신호 관찰 | 모든 신호 (VCD) | 포트에 나온 신호만 | 매 클럭 관찰 불가 |

세 가지를 다 해 보고 나니 에뮬레이션을 왜 하는지 조금 알 것 같다. 테스트벤치는 상대가 Verilator 모델인지 FPGA 회로인지 모른다. 시뮬레이션은 시뮬레이터가 베릴로그를 해석한 결과를 보여 줄 뿐이고, 에뮬레이션은 합성 도구가 실제로 만든 회로를 같은 테스트벤치로 시험한다. 두 결과가 같아야 시뮬레이션에서 본 동작이 실제 회로에서도 맞다고 할 수 있다. 프로토타이핑은 매 클럭을 들여다보지 않는 대신, 실제에 가까운 속도로 돌려 놓고 사람이 직접 게임을 하면서 확인한다.

## 6. 남은 의문

- Pico 가 클럭을 왜 100kHz 로 만드는지 아직 모르겠다. 펌웨어는 FPGA 가 내보내는 clk_o 가 올라갈 때마다 인터럽트로 좌표 13 비트와 화소 값을 digitalRead 로 하나씩 읽어 OLED 에 찍는다. 이 인터럽트를 처리할 시간 때문에 클럭을 더 올리지 못하는 것 같은데, 미니 오실로스코프로 GPIO28 을 재서 확인해 보려고 한다(블로그 과제 13).
- Q.13 의 에뮬레이션 모델은 클럭 상승 에지에서만 FPGA 와 주고받는데, Q.14 는 하강 에지에서도 주고받도록 바뀌었다. 통신이 두 배로 늘어나는 만큼 얻는 게 있을 텐데 그 이유를 아직 모르겠다.
- 에뮬레이션에서는 포트로 나온 신호만 보인다. 시뮬레이션 파형으로 보던 cannon_x 같은 내부 신호를 FPGA 에서 볼 방법이 궁금하다. 상용 에뮬레이터는 이걸 어떻게 해결하는지 찾아볼 생각이다(블로그 과제 11).
- FPGA 에서 14 퍼센트를 쓰는 이 회로가 1.9mm x 1.9mm 칩에서는 얼마나 차지할지 직접 합성해서 비교해 보고 싶다.

## 7. 내 칩으로 만들어 보고 싶은 것

요즘 관심을 갖고 보는 연구실이 있다. MIT Media Lab 의 Signal Kinetics 그룹(Fadel Adib 교수)이다. 바다처럼 관측이 어렵고 배터리를 갈아 주기 힘든 곳에서, 소리 에너지를 모아 쓰고 백스캐터로 통신하는 수중 센서 노드를 만든다. 노드는 신호를 직접 만들어 보내지 않고, 압전 소자에 연결된 스위치를 켜고 끄면서 들어온 음파를 반사하는 방식으로 데이터를 보낸다.

이 그룹의 2026 년 논문(Bringing All Modulations to Underwater Backscatter via PDM-Synthesis)은 스위치를 켜고 끄는 패턴만으로 CSS, QAM, OFDM 같은 복잡한 변조를 만든다. 노드의 STM32 가 2 차 델타시그마 변조를 소프트웨어로 계산해서 PDM 비트열을 만들고, 그 비트열로 스위치를 구동한다. 논문은 이 부분을 전용 하드웨어로 옮기면 전력을 더 줄일 수 있을 것이라고 후속 과제로 적어 두었다. 이걸 내 칩으로 만들어 보면 재미있을 것 같다.

연산량부터 따져 봤다. 기저대역 1kHz 에 오버샘플링 비율 100 이면 PDM 출력은 1 초에 20 만 비트다. 2 차 델타시그마 변조기 자체는 누산기 두 개와 비교기 하나 정도라 회로가 작고, 이번에 FPGA 에 올린 게임보다도 훨씬 작을 것이다. 문제는 계산이 아니라 입력이다. 변조기만 칩에 넣으면 MCU 가 입력 파형을 계속 밀어 넣어야 하고, 그동안 MCU 는 잠들 수 없다. 반대로 칩이 파형을 직접 만들게 하면 MCU 는 쉴 수 있지만 칩이 커진다. 이번 주 프로토타이핑에서도 Pico 가 클럭마다 인터럽트로 데이터를 읽느라 클럭을 올리지 못하는 것으로 보였는데, 같은 종류의 문제라고 생각한다.

그래서 생각하고 있는 구조는 MCU 가 파형의 대략적인 값만 가끔 넘겨주고, 칩이 그 사이를 채우거나 CSS 처럼 규칙이 있는 파형은 직접 만들어서 PDM 까지 내보내는 것이다. 그러면 MCU 는 대부분의 시간을 잠들어 있을 수 있다. 어느 쪽이 나은지는 칩 하나의 전력이 아니라 노드 전체가 비트 하나를 제대로 보내는 데 쓰는 에너지로 판단해야 한다. 변조 성능(검출률, 오류율, 처리량)이 나빠지면 전력이 줄어도 의미가 없다.

걸리는 점도 있다. 우리 수업의 ETRI 0.5μm 공정은 5V 로 동작해서, 1.8V 나 3.3V 로 도는 MCU 와 연결하려면 레벨 시프터와 전원 변환이 필요하다. 논문의 STM32 전력(평균 850μW)은 1.8V 기준이라 내 칩의 숫자와 그대로 비교할

수 없고, 이런 부가 회로의 전력까지 더해서 따져야 한다. 우선은 2 차 델타시그마 변조기를 베릴로그로 만들어 SystemC 테스트벤치에서 출력 스펙트럼을 확인하고, 이번 주에 해 본 것처럼 FPGA 에뮬레이션으로 검증한 다음, ETRI 표준 셀로 합성해서 면적을 확인해 보려고 한다.

## 마치며

시뮬레이션 화면으로만 보던 게임이 처음으로 실제 회로에서 돌았다. 코드보다 도구 버전, 칩 ID, 클럭 주파수처럼 코드 밖의 조건 때문에 막힌 시간이 더 길었다. 주석에 적힌 1MHz 를 그대로 믿었다가 틀린 것도 이번에 알았다. 다음부터는 숫자를 적을 때 어디서 확인한 숫자인지 같이 적어 두려고 한다.